

The **pkm** package manager complements the igos-build system nicely. It provides a lightweight, transparent, and reasonably safe way to manage the .igos.tar.gz archives produced by the build system. The hybrid SQLite + text manifest approach is pragmatic for a from-source distro — fast queries for everyday use, human-readable artifacts for auditing and recovery.

The CLI feels natural ("pkm provides /usr/bin/gcc", "pkm files bash"), and the core operations (install/remove/verify) align well with the build system's tracking. Overall, it's a solid foundation for Phase 1 of a custom package manager, though it has some security and robustness gaps typical of early custom PMs.

I'll address your specific points first, then broader feedback.

1. SQL Injection

Good news: All database interactions use **parameterized queries** with ? placeholders and tuple parameters (e.g., WHERE name = ?, (name,)). This is the correct and secure way to use sqlite3 in Python. No string concatenation or formatting of user-controlled values into SQL appears anywhere.

- search() uses LIKE ? with %term% — safe, as the wildcards are added in Python, and the term itself is bound.
- All other queries (insert, delete, select by name/path/id) are fully parameterized.
- No dynamic table/column names or raw executescript with user input.

Verdict: No SQL injection risks. This area is handled correctly.

2. & 5. File Operation Safety & Archive Extraction Security (Tar Slip / Path Traversal)

This is the weakest area.

Installation (installer.py):

- Archive extraction for inspection uses tar -xzf ... -C staging (no explicit path sanitization or filters).
- Deployment uses tar -xzf ... -C / --no-overwrite-dir --keep-directory-symlink — the GNU tar flags help with symlink handling, but they do **not** prevent malicious

archives from writing outside the target or abusing traversal (`../`, absolute paths, symlinks pointing outside, etc.).

- The "dangerous" top-level check (`lib/bin/etc.`) is the same smart safeguard as in the build system, but it only runs after extraction to a temp dir and only catches a few cases.
- No member-by-member validation, no rejection of absolute paths, no normalization to ensure everything stays under the root/staging.

Removal (`remover.py`): Uses direct `os.remove()` / `os.rmdir()` on paths from the database — safe as long as the DB is trustworthy, but TOCTOU is possible (file replaced between DB read and removal).

General risks:

- Archives come from the trusted build system (igos-build), so in normal offline use the risk is low.
- If an archive is ever supplied externally or tampered with, a tar-slip-style attack is possible (overwriting `/etc/passwd`, placing files in `/root`, etc.).
- Recent Python tarfile issues (even with filters) and historical CVE-2007-4559 highlight that blind `tar -x` is risky for any untrusted or semi-trusted input.

Recommendation:

- For the staging extraction, switch to Python's tarfile with `filter='data'` (Python 3.12+) or implement a safe extractor that normalizes paths and rejects anything with `..`, absolute paths, or suspicious symlinks.
- Or keep GNU tar but add `--anchored --no-wildcards` and pre-inspect members if possible.
- Consider running extraction in a very restricted environment (e.g., chroot or user namespaces) for extra defense, though that may be overkill for a chroot-based distro.

The current pre-deploy check is helpful but insufficient alone.

3. Config File Preservation

The logic in `remover.py` is mostly correct in intent:

- It identifies `/etc/` files (non-directories) via `is_config` flag set at install time.

- On removal, it recomputes the current SHA256 and compares it to the original_checksum stored in the config_files table.
- If they differ → preserve the file (skip os.remove).
- It also logs preserved files in the success message.

Issues / edge cases:

- Checksum is captured at **install time** (in add_files). If the package's post-install scripts or the user modify the file immediately after install, it may already be considered "modified" on first removal.
- The check runs SELECT inside the removal loop using self.db.conn directly — works, but it's better to go through a helper method for consistency.
- Directories under /etc/ are never removed (correct behavior).
- No handling for new config files created by the user after install (they won't be in the DB, so untouched — fine).
- Permission errors or missing files are silently skipped (best-effort removal is reasonable for a PM).

Verdict: It implements the common "don't nuke user-modified configs" pattern reasonably well (similar to Debian's conffiles or Slackware/RPM behaviors). It's not perfect (timestamp-based or diff-based alternatives exist), but checksum comparison is reliable for this use case. Document that checksums are from install time.

4. Dependency Resolution

Strengths:

- get_reverse_depends() correctly joins on the depends table to find packages that list this one as a dependency.
- Removal blocks unless --force is used, with a clear error message listing reverse deps.
- Dependencies are recorded at install time from the build system's metadata (build/host/runtime collapsed).

Limitations (not bugs, but scope):

- No automatic removal of "leaf" dependencies (like autoremove).
- No version constraints or "provides"/"replaces" (fine for LFS-style exact package names).

- Runtime vs build deps are not distinguished in reverse checks (everything blocks removal).

Verdict: The checker correctly prevents accidental breakage of dependents. Good safety net.

6. General Code Quality, Error Handling, Edge Cases, Maintainability

Positives:

- Clean separation: cli → database (core), installer/remover/verifier.
- Good use of argparse with subcommands.
- Hybrid DB + text manifests is excellent for transparency and recoverability (import_manifests is a nice migration path).
- History logging, verify (with checksums for non-config files), provides/files queries are useful.
- Best-effort removal and detailed error messages.
- WAL journal mode and foreign keys are good SQLite practices.

Areas for improvement:

- **Error handling:** Many operations are "best effort" (e.g., removal ignores OSError). This is pragmatic but can hide real problems. Consider collecting errors and reporting them.
- **TOCTOU races:** Between listing files and acting on them (install/remove/verify). Rare in single-user root context, but present.
- **Root assumption:** Everything assumes running as root and targeting /. The --root support in PackageInstaller exists but is under-used in CLI (no argument passed from cli.py).
- **Edge cases:**
 - Packages with no files or only directories.
 - Archives with symlinks, hardlinks, devices, setuid bits.
 - Very large packages (memory usage in file listing).
 - Conflicting package names or duplicate files across packages (DB allows it via OR REPLACE, but ownership query returns only one).
 - import_manifests parsing is regex/heuristic-based — fragile for malformed manifests.
- **Maintainability:**

- Some duplication between build system and pkm (manifest writing, dangerous-path checks, size calculation).
- Magic paths (/var/lib/igos/...) are repeated; consider centralizing in a config.
- No unit tests visible — critical for DB logic, safe extraction, and checksum handling.
- verify uses checksums only for non-config files (sensible), but could warn on config drift separately.
- **Missing features** (noted in code): list available / update / upgrade — planned for Phase 2.

Security summary (beyond extraction):

- Low risk in normal use (offline, root-run, archives from trusted build).
- Run as non-root where possible for queries (but install/remove need root).
- Consider signing archives in the future for stronger integrity.

Overall Thoughts & Recommendations

pkm is a thoughtful, minimal package manager that fits the InterGenOS philosophy: reproducible, auditable, and not overly complex. It reuses concepts from the build system (manifests, dangerous-path checks, tracking) effectively. The natural-language CLI is a nice touch.

Priority fixes:

1. **Harden archive extraction** — Use filtered tar handling (Python tarfile with filter='data' + path validation, or stricter GNU tar flags + member inspection). This is the biggest remaining risk.
2. Expose --root in CLI and make installer/remover fully respect it (including DB path, manifests, etc.).
3. Centralize constants and add more logging for failures.
4. Add basic tests for DB operations, config preservation, and a safe_extract helper.
5. Document assumptions (root, offline, trusted archives) and the hybrid DB/manifest model.

Minor polish:

- Make checksum computation optional or configurable (expensive on huge packages).

- In remover, use a transaction for DB cleanup.
- Consider adding reinstall / downgrade commands.

Compared to the build system review, pkm feels slightly less mature (especially extraction safety), but the foundation is strong. Once extraction is hardened and --root is fully wired, this will be a reliable companion to igos-build.

If you share example .igos.tar.gz contents, a sample build.sh, or specific build.sh examples from packages, I can review how well the two systems integrate in practice. Or let me know if you'd like refactored snippets for safe tar extraction or other improvements!